Mira: AI Project Intelligence Assistant

Capstone submission, Q1 to Q4 · Danish · Applied Agentic AI for PMs/TPMs · Repo: github.com/danihak/mira-ai-project-assistant

Q1 — Ideation: AI Agents for PM/TPM Productivity at Nexora

Author: Danish

Context

Nexora Pvt. Ltd. (London, 120+ people, clients across Europe and Asia) runs several client projects at once with only a few PMs/TPMs, who also wear other hats. It has just taken on the 6-month AI Adoption Project for ABCDE Ltd. (logistics). The brief names six pain points:

- 3–4 hours to build each project plan
- inconsistent risk assessments
- 1–2 hours per weekly status report
- milestones missed for lack of alerts
- no standard formats
- too few PMs for the number of projects

These three use cases target the biggest time sinks and the biggest trust risks. All three are built into Mira.

Use case 1 — Project Plan Generator

Pain point	A PM spends 3–4 hours turning a client's high-level description into a phased plan. Every PM uses a different format, so plans are hard to compare or review, and early plans often carry over unstated assumptions.
Agentic solution	The **Planner Agent** reads the project description and timeline. Code works out the real calendar dates for each phase. The agent writes a 7-section plan (summary, goals, phase table, milestones, team, dependencies and assumptions, open questions) and links existing board tasks to phases. Inferences are tagged `[ASSUMPTION]`. If the input is vague, the rule-based Clarifier asks for scope, deliverables, timeline and team instead of inventing them.
Inputs	Project description (TXT/brief), phase timeline (CSV), task board export (optional), project start date
Outputs	A standard-format Markdown plan, ready for the PM to review and send to the client. Open questions for the sponsor are listed separately.
Success metrics	1) **Plan groundedness:** 100% of milestones traceable to the timeline or description. 2) **Time to first draft:** from 3–4 hours to under 15 minutes including PM review. 3) **Edit**

	distance / acceptance: at least 80% of generated sections accepted with only light edits.
Knowledge base needed	The client's project description and SOW, a timeline template, Nexora's standard plan template, past plans for similar projects (for retrieval or fine-tuning), and a glossary of the client's domain.

Use case 2 — Weekly Status & Stakeholder Reporting

Pain point	Every week a PM spends 1–2 hours exporting the board, counting tasks, chasing owners and rewriting the same facts for leadership, engineering and the client. Manual counts drift, and reports go out late or not at all.
Agentic solution	**Compute Facts** (code) counts tasks by status and sprint and finds overdue and blocked items. The **Status Reporter Agent** formats this into a weekly report with a rule-based RAG health rating. The **Stakeholder Update Agent** writes an email for one sprint, in a tone set for the audience. Each agent only sees the sprint in scope, so other sprints can't leak into the email.
Inputs	Task board export (CSV, or Trello/Jira API), sprint or week requested, audience, report date
Outputs	Weekly status report (Markdown), stakeholder email, and the exact counts as JSON for dashboards
Success metrics	1) **Status accuracy:** 100% of counts match the board. 2) **Reporting time:** from 1–2 hours to under 10 minutes a week. 3) **On-time reporting:** at least 95% of weekly reports sent by Friday.
Knowledge base needed	Live task board (Trello/Jira) or exports, sprint calendar, stakeholder list with each person's preferred format, and Nexora's report template and RAG-rating definitions

Use case 3 — Risk & Milestone Early Warning

Pain point	Risk registers are written at kick-off and never connected to what is actually happening. Milestone slips are found in retrospectives instead of being prevented. Some projects have no risk matrix at all.
Agentic solution	The **Risk Assessor Agent** combines the project's risk register with live delivery signals. For example, a blocked security review (T024) raises R09 Security, and an open legal review (T025) raises R05 Ethical & Legal. The result is a risk matrix with an evidence column. The **Milestone Tracker Agent** flags blocked, overdue and at-risk tasks and milestones due in the next 14 days, using documented rules. These are the same checks a scheduled daily run would perform.
Inputs	Risk register (CSV), timeline with milestones, task board, report date, alert windows (7 and 14 days)
Outputs	Evidence-based risk matrix, top-N risk answer, and an alerts table (Blocked / Overdue / At risk / Upcoming

	milestone) with suggested actions
Success metrics	1) **Risk relevance:** 100% of risks come from the register or are evidenced by a task, with no generic risks. 2) **Early detection:** at least 80% of slipped milestones flagged at least 7 days before their due date. 3) **Risk coverage:** 100% of active projects have a current risk matrix, up from ad hoc.
Knowledge base needed	Project risk register, Nexora's risk taxonomy (categories and ratings), timeline, live board, and a history of past project risks and incidents (for retrieval and calibration)

Why these three, and the order to build them

1. **Status reporting first.** It's weekly and high-volume, its accuracy is easy to check against the board, and it gives the fastest return.
2. **Milestone alerts second.** They reuse the same computed facts and turn reporting into prevention.
3. **Planning and risk third.** They are less frequent but carry the highest stakes, so they need a PM review step before anything is shared with the client.

Across all three, the design principle is the same: **code computes, the LLM explains, and Mira asks when it doesn't know.**

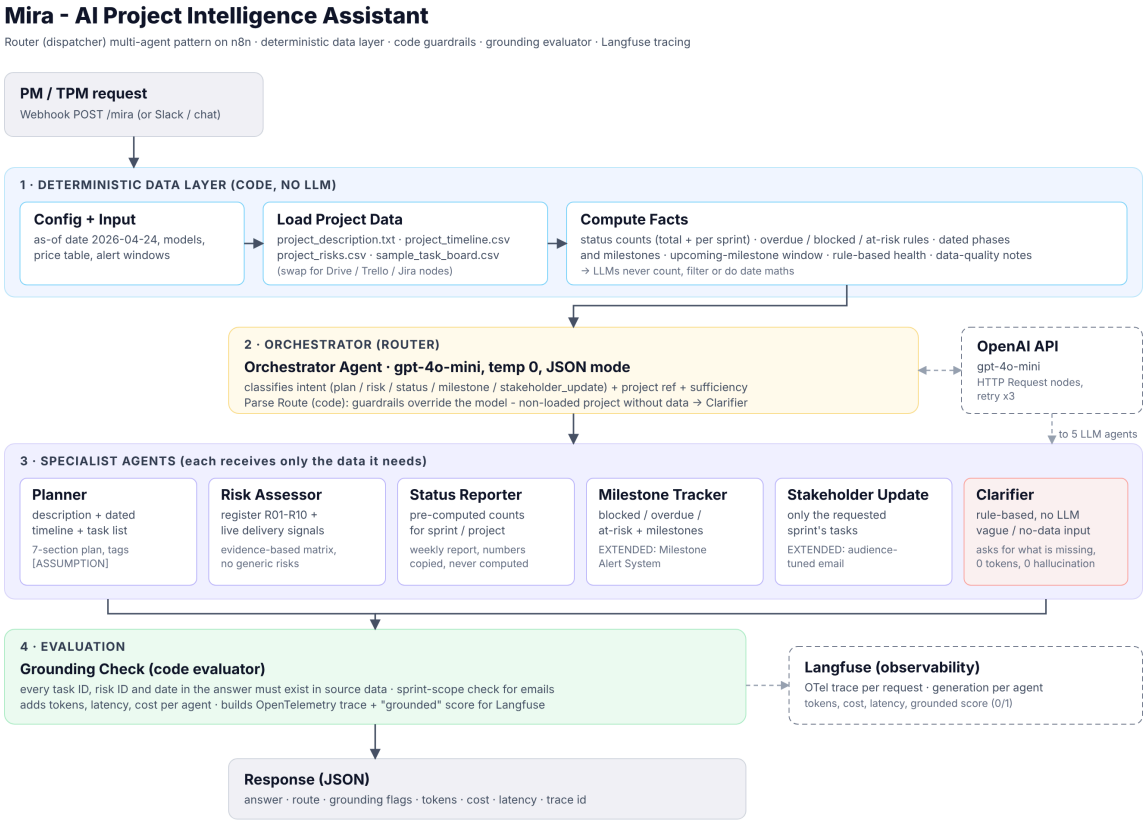
Q2 — Mira: Architecture, Implementation & Evaluation Write-up

Author: Danish · **Capstone:** Applied Agentic AI for PMs/TPMs — Mira (Nexora)

1. Problem summary

Nexora's PMs/TPMs run 3–5 projects at once and spend 3–4 hours writing each project plan, 1–2 hours a week compiling status reports from task boards, and produce risk assessments that vary from person to person. Mira is a multi-agent assistant that generates project plans, risk assessments, weekly status reports, milestone alerts and stakeholder updates from a project's own files. The biggest risk is fabrication — invented milestones, task statuses or generic risks that a PM forwards to leadership — so Mira is designed so the LLM never has to compute or guess anything.

2. Architecture



Mira architecture

Flow: Webhook → Config + Input → Load Project Data → **Compute Facts (code)** → Orchestrator Agent (LLM) → Parse Route (code guardrails) → Switch → one of 5 specialist agents or the rule-based Clarifier → Package Response → **Grounding Check (code evaluator)** → Langfuse → Response.

3. Tool selection

Category	Choice	Why
Workflow platform	**n8n**	Visual multi-agent canvas, native Webhook/Switch/Code/HTTP nodes,

Category	Choice	Why
		and I already run an n8n + Slack bot, so Mira can be exposed in Slack with no new platform.
LLM	**gpt-4o-mini** for all agents (configurable per agent in one node)	Routing and formatting pre-computed facts do not need a frontier model. ~$0.001 per request. Planner/Risk can be switched to gpt-4.1 / gpt-4o in Config if quality demands it.
LLM integration	**HTTP Request → OpenAI Chat Completions** (not the AI Agent node)	Full control of temperature and JSON mode, and the raw `usage` field gives exact tokens per agent for cost tracking and tracing.
Data	Project CSV/TXT files parsed in a Code node	Works on n8n Cloud and self-hosted; swap for Google Drive / Trello / Jira nodes later without touching the agents.
Observability	**Langfuse** (OpenTelemetry endpoint + Scores API)	One trace per request with typed observations (agent → span → generation → evaluator), tokens, cost and latency per agent, plus a `grounded` score. I first used the v3 ingestion API, but its response warned it is retired on 16 Nov 2026 and delays data, so I migrated to OTLP before submission. Free Hobby tier (50k units/month) covers evaluation and a 10-PM pilot.

4. Orchestration pattern — Router / Dispatcher

Mira's inputs are **different request types that need different data and different output formats** (plan, risk, status, milestone tracking, stakeholder email). A sequential pipeline would run every agent on every request, adding cost and giving agents irrelevant data to hallucinate from. A hierarchical pattern is unnecessary because no test request needs several agents to collaborate. So: one cheap Orchestrator classifies the request, and exactly one specialist answers it.

Two design decisions make the router reliable:

1. **Code overrides the model.** The Orchestrator returns JSON (`intent`, `project_ref`, `sprint`, `sufficient`, `missing_info`). The *Parse Route* code node then applies hard rules: a status/milestone/stakeholder request for a project with no loaded data, or a plan/risk request that is not sufficient, always goes to the Clarifier — whatever the model said.
2. **Vague inputs never reach an LLM.** The Clarifier is rule-based: it lists exactly what is missing for that request type. Zero tokens and zero chance of a "helpful" invented plan.

5. Agents

Agent	Input (only what it needs)	Output	Key prompt rules
Orchestrator (LLM, temp 0, JSON)	User request + list of loaded data	Intent, project ref, sprint, sufficiency, missing info	Strict sufficiency rules; "at risk of missing deadline" =

Agent	Input (only what it needs)	Output	Key prompt rules
			milestone, not risk
Planner (LLM)	Description, timeline with code-computed dates, task list	7-section plan: summary, goals, phase table, milestones, team, dependencies, open questions	No invented goals/dates/people; inferences tagged `[ASSUMPTION]`; flag board/timeline mismatches
Risk Assessor (LLM)	Risk register R01–R10 + delivery signals (blocked, overdue, at-risk, security/compliance/vendor tasks)	Risk matrix with evidence column, or ranked answer to a question	Only R01–R10; no generic risks; mitigation copied from register; "No current signal" when none
Status Reporter (LLM)	Pre-computed counts and task lists for the scope	Weekly report or a direct count answer	Every number copied from FACTS; no percentages not in FACTS
Milestone Tracker (LLM) — *extended*	Dated phases, milestones in the 14-day window, blocked/overdue/at-risk tasks with reasons	Direct answer + alerts table + outlook	Only timeline milestones; "Why" must quote the computed reason
Stakeholder Update (LLM) — *extended*	**Only** the requested sprint's tasks + audience	Professional email	Out-of-scope sprints are never in its context, so they cannot leak
Clarifier (code)	Router output	What's missing + what Mira can do	Deterministic
Compute Facts (code)	All project files	Counts, dates, flags, health	All arithmetic and filtering lives here
Grounding Check (code)	Answer + source facts	Pass/flags, cost, Langfuse batch	Every task ID, risk ID and date cited must exist; sprint-scope check

Reference date. The sample board runs April–August 2026 with Sprint 3 active, so Mira reports *as of 24 Apr 2026* (end of timeline week 4; week 1 starts 30 Mar 2026, consistent with the Sprint 1 due dates). Without a fixed date, "next 2 weeks" and "at risk" are undefined and the LLM would guess. The date is one setting in *Config + Input*, or can be passed per request.

At-risk rules (documented, deterministic): Blocked = status Blocked · Overdue = not Done and past due · At risk = In Progress due within 7 days, or To Do due within 14 days. **Health:** RED if ≥3 blocked+overdue, AMBER if ≥1, else GREEN.

6. Cost analysis

Measured on the live run (n8n Cloud, gpt-4o-mini, 12 baseline inputs, Run 2). Mira returns per-agent tokens and cost in every response. Prices (OpenAI list, per 1M tokens): gpt-4o-mini $0.15 in / $0.60 out.

Test	Route	Tokens in / out	Cost	Latency
T1 plan	Planner	3,883 / 1,036	$0.00120	10.9 s
T3 risk assessment	Risk Assessor	3,532 / 916	$0.00108	10.6 s

Test	Route	Tokens in / out	Cost	Latency
T7 top-3 risks	Risk Assessor	3,530 / 294	$0.00071	3.3 s
T5 sprint status	Status Reporter	2,545 / 447	$0.00065	4.4 s
T10 status counts	Status Reporter	4,067 / 158	$0.00071	2.3 s
T8 / T11 milestones	Milestone Tracker	2,842 / 632	$0.00080	6.3–7.9 s
T12 stakeholder email	Stakeholder Update	1,452 / 224	$0.00035	3.3 s
T2/T4/T6/T9 vague	Clarifier (router only)	~772 / ~95	$0.00017	1.3–1.9 s
All 12			**$0.0070 total · $0.00058 avg**	p50 3.3 s · max 10.9 s

The estimates made before the run, using exact prompt tokens and estimated outputs, were within about 10% of these measured values.

Request type	Router in/out	Specialist in / out (est.)	Cost per request
Project plan	779 / 70	3,105 / 1,100	$0.0013
Risk assessment	779 / 70	3,262 / 1,200	$0.0014
Sprint status report	775 / 70	1,772 / 700	$0.0008
Project status counts	780 / 70	3,289 / 300	$0.0008
Milestone / blocked tasks	776 / 70	1,948 / 500	$0.0008
Stakeholder email	771 / 70	682 / 300	$0.0004
Vague input → Clarifier	778 / 70	0 / 0 (code)	$0.0002

Cost item	Estimate
Average cost per request (measured: $0.00058 across the 12 tests; $0.00079 for LLM-answered requests)	~$0.0008
Requests per PM per day	15 (e.g. 3 status, 4 milestone checks, 2 stakeholder updates, 2 risk questions, 1 plan/week, 3 ad-hoc incl. vague)
Cost per PM per day	**~$0.012** → **~$0.26 per month** (22 working days)
Same mix with gpt-4o for all specialist agents	~$0.01 per request → ~$3.30 per PM per month
Data/API calls	CSV parsing in n8n — no API cost; Trello/Jira APIs free at this volume
Langfuse	Free Hobby tier (50k units/month): ~6 events per request × 15 × 22 ≈ 2k per PM per month; a 10-PM pilot ≈ 20k
Pilot total (10 PMs)	**≈ $3/month LLM** + n8n hosting

Per plan: ~$0.0013 versus 3–4 hours of PM time. The cost driver is input context, not output — which is why each specialist gets only its slice of data.

7. Evaluation strategy — production metrics

#	Metric	Definition	Target	How measured
1	**Groundedness rate**	% of responses where every cited	≥ 98%	Automatic: Grounding Check

#	Metric	Definition	Target	How measured
		task ID, risk ID and date exists in source data		`grounded` score in Langfuse
2	**Status accuracy**	% of status reports whose counts match the task board	100%	Counts come from code; spot-check weekly against board export
3	**Correct refusal rate**	% of vague / no-data requests routed to the Clarifier (no plan, risk or status invented)	100% on the vague test set	Langfuse filter `route:clarify` vs labelled vague inputs
4	**Plan groundedness**	% of milestones in generated plans traceable to the timeline	100%	Test-runner check + monthly sample review
5	**Time saved / adoption**	Minutes saved per status report and plan; weekly active PMs	≥ 60 min/PM/week; 8 of 10 pilot PMs weekly	Before/after survey + n8n execution counts
6	**Cost & latency**	USD and p95 latency per request	< $0.002, p95 < 15 s	Langfuse dashboards

8. Testing observations

Found while building (before the LLM run):

- **The problem statement's sample-data note is wrong.** It says the board has "3 Done … the rest To Do". The CSV actually has **5 Done (T001–T005), 3 In Progress (T006, T007, T025), 1 Blocked (T024), 16 To Do**. Because counts come from code, Mira reports the real numbers; an LLM "anchored" on the brief would fail T10.
- **The board and the timeline don't fully line up.** The last task is due 4 Aug 2026 but timeline week 24 ends 13 Sep 2026, and several deployment tasks fall before their phase window. Mira surfaces this as a data note instead of silently "fixing" dates.
- **Router ambiguity → fixed.** "Which tasks are … *at risk* of missing their deadline" (T8) collides with the *risk* intent. Added an explicit disambiguation rule to the Orchestrator prompt (task delivery = milestone; risk register = risk), and the mock-router regression run now routes T8 correctly.
- **T006 is the hidden test.** Besides the deliberately blocked T024, T006 (Sprint 2, due 21 Apr) is still In Progress as of 24 Apr, so it is overdue and is also the correct "carry-over" in the Sprint 2 stakeholder email.
- **Grounding Check verified** with an injected hallucination (fake task T099, risk R13, an unknown date and an out-of-sprint task): all four were flagged.

Live baseline runs (n8n Cloud, gpt-4o-mini). Results are in `tests/results/` (`run1_baseline_results.md` and `baseline_results.md`).

	Run 1 (v1)	Run 2 (v2, after fixes)
Automatic keyword checks	11/12 (T8 failed)	**12/12**
Routed correctly	12/12	12/12
Vague inputs refused (T2, T4, T6, T9)	4/4	4/4
Grounding check passed	12/12	12/12
Manual review issues	2 (T7, T8)	0
Total cost of 12 tests	$0.0074	$0.0070

What went wrong in Run 1, and the fix (the evaluate → fix → re-evaluate loop):

1. **T8 omitted a task.** The Milestone Tracker said "4 tasks at risk" but listed only 3 (it dropped T009). It also named tasks without IDs, so "T024" never appeared literally. **Fix:** code now passes `alert_counts`; the prompt requires the task ID in every row and a row count equal to `total_task_alerts`. **After:** all 6 alerts listed with IDs (T024, T006, T007, T008, T009, T025).
2. **T7 linked evidence to the wrong risk.** The blocked *security* review (T024) was cited as evidence for R02 *Integration*, and R04 was justified with "ongoing tasks related to stakeholder engagement", which cites no task. The top-3 also disagreed with T3. **Fix:** risk-to-task evidence links and the ranking are now computed in code (security → R09, compliance/legal → R05, data → R01, vendor/platform → R10, and so on; score = Blocked 30 / Overdue 20 / At risk 10 + priority). The agent may only quote those links. **After:** R09 (T024 blocked) → R01 (T006 overdue) → R05 (T025 at risk), with consistent evidence, and the answer is 70% shorter ($0.0012 → $0.0007).
3. **The evaluator had a blind spot.** The Grounding Check passed both failures, because it detected *invented* IDs but not *omitted* or *mislinked* ones. **Fix:** added completeness checks (every alert task in milestone answers, every in-scope task in sprint reports and emails). Mislinked evidence is now prevented by construction rather than detected afterwards.

9. Evaluation in Langfuse: dataset, experiments, LLM judge, dashboard

- **Dataset `mira-baseline`:** the 12 baseline inputs and their expected criteria are stored as Langfuse dataset items `mira-T1` … `mira-T12`. They are created once by the `workflow/experiments/langfuse_dataset_setup.json` n8n workflow.
- **Experiments, v1 vs v2:** the same 12 items were run live through two deployed workflows: **v1** (original Risk and Milestone prompts, `workflow/experiments/v1/`) and **v2** (current). Both use the same evaluator. Each trace is linked to its dataset item and run, so Langfuse compares them side by side.
    - **Result:** v1 reproduced the original failures on a fresh run. T8 again omitted T009, T11 omitted three alerts, and T7 again cited the blocked security review as evidence for *Integration* risk. v2 passed all 12. Both runs cost about $0.007.
- **LLM-as-a-judge (`faithfulness`):** gpt-4.1-mini, a different and stronger model than the generator, scores every specialist answer from 0 to 1. It checks the answer against the exact source data the agent received (the generation's input). It runs automatically on all five specialist agents (not the Orchestrator) at about $0.0016 per check. It covers what the code evaluator cannot see: mislinked evidence and unsupported reasoning.
- **Dashboard "Mira - Production Metrics":**

    - groundedness and faithfulness averages, and their trend over time
    - p50/p90/p95/p99 latency per agent
    - cost per agent, including the judge
    - request mix by agent

- **Evaluating the evaluators.** On the three defects v1 reproduced, I checked each automatic evaluator against the real answer text:

v1 defect	Code Grounding Check	LLM judge (`faithfulness`)	Truth (manual check)
T8 omits at-risk task T009	**Caught** ("Omitted alert task T009")	**Missed (1.00)**; it claimed T009 was listed	T009 absent
T11 omits 3 alerts	**Caught**	**Caught (0.70)**	Omitted
T7 cites the security blocker as *Integration* evidence	Not designed to catch	**Missed (1.00)**: "integration and security are related"	Wrong link

  Lesson: the LLM judge can itself hallucinate (T8) and rationalise plausible-sounding errors (T7). Code checks are more reliable for anything countable. The judge adds value for reasoning-level issues, but it needs its own evaluation, ideally with a stronger judge model and a labelled set of known-bad answers. The T7 class of error is prevented by design in v2: evidence links are computed in code, not judged afterwards.

- **Finding from the dashboard:** the LLM judge costs more than all of Mira's agents combined. In production I would sample it (for example 10–20% of traffic, plus 100% of anything the code evaluator flags) rather than judge every answer.

Q3 — Program Charter: Mira, AI Project Intelligence Assistant

Program owner: Danish (AI PM/TPM) · **Sponsor:** CTO, Nexora Pvt. Ltd. · **Version:** 1.0

1. Vision

Every Nexora PM/TPM can get an accurate plan, risk view and status report for any project in minutes. Every number in those outputs traces back to the project's own data, and PMs spend the hours they get back on clients and delivery, not paperwork.

Problem statement. Nexora runs several client projects with a handful of PMs/TPMs. Plans take 3–4 hours each, weekly status reports take 1–2 hours, risk assessments vary by person, and missed milestones are only discovered in retrospectives. There is no standard format across the team.

2. Objectives

1. Cut the time to produce a project plan from 3–4 hours to **under 15 minutes** of PM review.
2. Cut weekly status reporting from 1–2 hours to **under 10 minutes** per project.
3. Give **every active project** a current, evidence-based risk matrix.
4. Flag at-risk milestones **at least 7 days before** they are due.
5. Standardise plan, status and risk formats across all PMs.

3. Scope

In scope (MVP, this program)	Out of scope (later / never)
Project plan generation from a description and timeline	Writing directly back to Jira/Trello (Mira is read-only in the MVP)
Risk assessment and top-risk Q&A grounded in the risk register	Resource and capacity planning (extension candidate)
Weekly status report and task-count Q&A from the board	Budget and financial forecasting
Milestone alert system (blocked, overdue, at risk, upcoming)	Sending email to clients without a human (always PM-approved)
Stakeholder update emails per sprint and audience	Automated people-performance judgements (never)
Webhook API, Slack front end, Langfuse tracing, grounding evaluator	Multi-language outputs
Pilot on the ABCDE Ltd. AI Adoption Project, then 10 PMs	Fine-tuned models (Phase 3, only if the evaluation shows a need)

Assumptions: PMs keep task boards current. Client data may be processed by OpenAI under Nexora's API data-processing terms. n8n is hosted by Nexora. **Constraints:** LLM spend under $50/month for the pilot. No client data stored outside approved systems. UK GDPR applies.

4. Success criteria (go-live gates)

Area	Metric	Target
Trust	Groundedness rate (all cited IDs and dates exist in source)	≥ 98%
Trust	Correct refusals on vague or no-data inputs	100% on the test set
Accuracy	Status counts match the board	100%
Accuracy	Baseline test suite (12 inputs)	12/12 pass before pilot; no regression on each release
Value	Time saved per PM per week	≥ 60 minutes (survey + time log)
Adoption	Pilot PMs using Mira weekly	≥ 8 of 10 by week 4 of the pilot
Cost	LLM cost per request / per PM per month	< $0.002 / < $1
Performance	p95 latency	< 15 s

5. Timeline (12 weeks)

Phase	Weeks	Key activities	Exit milestone
0. Build & evaluate (capstone)	1–2	Router + 5 agents, grounding evaluator, Langfuse, baseline test suite	12/12 baseline tests pass; architecture signed off by CTO
1. Harden	3–4	Trello/Jira connectors, Slack front end, PM review step, prompt fixes from traces	Security & privacy review passed
2. Pilot (ABCDE project)	5–6	2 PMs use Mira on the live ABCDE project; weekly reports compared side by side with manual ones	Accuracy 100%, groundedness ≥ 98%, PM sign-off
3. Pilot expansion	7–10	10 PMs across 3–5 projects; scheduled daily milestone alerts; collect edits for a fine-tuning dataset	Adoption ≥ 8/10, time saved ≥ 60 min/week
4. Go / no-go & scale	11–12	Review metrics, cost, incidents; decide on company-wide rollout and the Phase-2 backlog	Steering committee decision

6. Stakeholders & RACI

Stakeholder	Interest	Plan	Build	Pilot	Go/No-go
CTO (sponsor)	Productivity, risk, cost	A	I	I	**A**
AI PM/TPM — program owner (Danish)	Delivery of Mira	R	A	A	R

Stakeholder	Interest	Plan	Build	Pilot	Go/No-go
ML / AI engineer	Prompts, evaluation, fine-tuning	C	R	C	C
Software developer (n8n, integrations)	Connectors, hosting	C	R	C	I
PMs/TPMs (users)	Time saved, trust in outputs	C	C	R	C
IT security & Data Protection Officer	Client data, UK GDPR, vendor terms	C	C	C	C (veto on privacy)
Delivery heads	Consistent reporting across accounts	I	I	C	C
Client — ABCDE Ltd. PMO	Receives Mira-assisted reports	I	–	I	I

R = Responsible · A = Accountable · C = Consulted · I = Informed

7. Decision-making & governance

- **Steering committee** (CTO, program owner, a delivery head, DPO) meets every two weeks and decides scope changes, phase gates and spend above budget.
- **Weekly working session** (program owner, engineers, 2 pilot PMs) reviews the Langfuse dashboard (groundedness, failures, cost) and agrees prompt or rule changes.
- **Decision rights:** the program owner decides prompts, models within budget and backlog order. The CTO decides phase gates and go/no-go. The DPO can veto any change to the data flow.
- **Change control:** every prompt or model change re-runs the 12-input baseline suite. A regression blocks release.
- **Human-in-the-loop policy:** nothing Mira generates goes to a client or leadership without PM review. Grounding flags are shown to the PM before sharing.

8. Risks & mitigations

#	Risk	L	I	Mitigation	Owner
1	Hallucinated milestones, statuses or risks reach a client	M	H	Counts and dates computed in code; context limited to what each agent needs; Grounding Check on every answer; PM approval before sharing	Program owner

#	Risk	L	I	Mitigation	Owner
2	Client data privacy (UK GDPR, client contracts)	M	H	DPO review; API data-processing terms with no training on data; send only the data each agent needs; no personal data beyond assignee names; logs kept for 30 days	DPO
3	Stale or inconsistent board data gives wrong reports	H	M	Data-quality notes surfaced (e.g. board vs timeline mismatch); “as-of” date on every report; PMs own board hygiene	PMs
4	Low adoption — PMs don’t trust or use it	M	H	Pilot with champions; side-by-side comparison with manual reports; Slack front end; collect and act on feedback weekly	Program owner
5	Model or vendor changes break outputs (drift)	M	M	Pinned model versions; regression suite on every change; model choice is one config setting	ML engineer
6	Cost overrun at scale	L	M	gpt-4o-mini by default, small per-agent context, rule-based Clarifier; cost tracked per request in Langfuse	Program owner
7	Vendor lock-in (OpenAI, n8n)	L	M	Plain HTTP calls to an OpenAI-compatible API (swappable); prompts and code kept in Git	Developer

9. Rollout plan

1. **Shadow mode (weeks 5–6):** 2 pilot PMs produce their reports both manually and with Mira; differences are logged and fixed.
2. **Assisted mode (weeks 7–10):** 10 PMs use Mira drafts and review them before sending. Daily milestone alerts post to each project's Slack channel. Weekly office hours are held.
3. **Default mode (week 11+, if the gate is passed):** Mira's report becomes the standard weekly format, and the PM review step stays.
4. **Enablement:** a 30-minute training session, a one-page "how to ask Mira" guide, a prompt library and a feedback button in Slack.
5. **Measurement:** a Langfuse dashboard (groundedness, refusals, cost, latency) plus a monthly time-saved survey reported to the steering committee.

10. Budget (pilot)

LLM about $3/month for 10 PMs at gpt-4o-mini (see the cost analysis in the Q2 write-up). Langfuse is on the free Hobby tier (50k units/month). The rest is n8n hosting on existing infrastructure plus about 2 weeks of engineering time for Phase 1.

Q4 — Reflection and Next Steps

Author: Danish

> Based on two live runs of the 12-input baseline on n8n Cloud with gpt-4o-mini (`tests/results/`).

1. What the traces showed

- **The biggest risk was in the data, not the model.** The brief describes the task board as "3 Done … the rest To Do", but the CSV has 5 Done, 3 In Progress, 1 Blocked and 16 To Do. Any design that lets the LLM count, or that prompts it with the brief's description, fails T10. Moving all counting and date maths into the *Compute Facts* code node removed this whole class of error.
- **Routing ambiguity is a real failure mode.** "Which tasks are … *at risk* of missing their deadline" looks like a *risk* request to a classifier. I added an explicit disambiguation rule (task delivery = milestone; risk register = risk) and re-ran the suite to confirm the fix.
- **Hidden edge case.** T006 (Sprint 2, due 21 Apr) is still In Progress on the report date, so it is overdue. The correct T8 answer contains two items, not just the deliberately blocked T024, and T006 is the honest "carry-over" in the Sprint 2 email.
- **Vague inputs cost almost nothing.** Because vague requests go to a rule-based Clarifier, T2, T4, T6 and T9 cost about $0.00015 each (router only) and cannot hallucinate. An LLM that "tries to help" on vague input produces longer, more expensive and invented output.
- **Run 1: 11/12; Run 2: 12/12.** All 12 requests were routed correctly in both runs, and all four vague inputs were refused, each costing $0.00017.
- **Failure 1, Milestone Tracker (T8):** it reported "4 at risk" but listed 3, silently dropping T009, and used task names without IDs. The per-agent trace showed the input FACTS were correct, so the agent was at fault, not the data. Fix: alert counts passed from code, plus a "one row per alert, ID first" rule. After the fix, all 6 alerts appear with IDs.
- **Failure 2, Risk Assessor (T7):** it cited the blocked security review as evidence for *Integration* risk and gave a vague "ongoing tasks" justification. Fix: evidence links and the ranking are computed in code, and the agent may only quote them. After the fix, R09 → R01 → R05 with correct evidence, at 40% lower cost.
- **Lesson about evaluators:** my own Grounding Check passed both failures. It caught *invented* IDs, not *missing* or *mislinked* ones. Adding completeness checks and manual review of every output was what exposed them. An automatic metric is only as good as the failure modes it was designed for.

2. Improvement plan

1. **Ask Mira questions in Slack.** Front-end the webhook with the existing n8n Slack bot, threaded per project. Acknowledge immediately to stay within Slack's 3-second limit.
2. **Live data instead of CSV.** Replace *Load Project Data* with Trello/Jira and Google Drive nodes. The agents don't change, because they only consume *Compute Facts*.
3. **Scheduled alerts.** Run the Milestone Tracker every morning per project and post only when something changed (new blocked or overdue item, milestone within 7 days).
4. **Human-in-the-loop.** Plans and stakeholder emails go to the PM for one-click approval before sending. Grounding flags are shown inline.
5. **LLM-as-judge (done).** A `faithfulness` judge (gpt-4.1-mini) now scores every specialist answer in Langfuse against the exact source data the agent saw. **Next:** sample it at 10–20% plus 100%

of code-flagged answers, because the dashboard shows it costs more than Mira itself.
6. **Multi-project support.** Key the data and facts by project ID so one PM can ask “which of my projects is red?”

3. Evaluation and fine-tuning

Experiments in Langfuse. The 12 baseline inputs are a Langfuse dataset. I re-ran them live through the original v1 prompts and the current v2, with the same evaluator. v1 reproduced its failures on a fresh run (T8 dropped T009 again; T7 again tied the security blocker to Integration risk), while v2 passed 12/12. Being able to reproduce a failure, then show the fix removing it on the same inputs, is the strongest evidence the fix is real and not luck.

The judge needs judging too. On those reproduced v1 defects, my code checker caught 2 of 3, and the LLM judge (gpt-4.1-mini) caught only 1 of 3. It scored the T8 answer 1.00 while claiming the missing task was present, and it accepted the security→Integration mislink as “related”. So: count with code, judge reasoning with an LLM, and test the judge on a labelled set of known-bad answers before trusting its scores.

The course loop is *evaluate → find failure → fix → re-evaluate*. Mira bakes the first step into every request: the Grounding Check records a `grounded` score (0/1) on every Langfuse trace, so groundedness becomes a production metric, not a one-off test. The 12 baseline inputs are the regression suite, and any prompt or model change must pass them again.

Fine-tuning comes **after** prompt and architecture fixes, and only for the agent that still fails. The most likely candidate is the Planner, which has the most free text and the most room to add “helpful” goals. The plan:

- Collect 50–100 PM-edited plans from the pilot as `system / user / assistant` JSONL, including about 15 vague-input examples whose correct answer is “insufficient detail”.
- Fine-tune gpt-4o-mini, swap the model ID in *Config + Input*, and re-run the same suite and the Langfuse groundedness metric to compare before and after.

If prompts plus code already achieve over 98% groundedness, fine-tuning isn’t worth the cost of maintaining it.

4. Privacy considerations

- **Data minimisation:** each agent receives only the slice it needs. For example, the Stakeholder agent sees one sprint, and no agent sees raw files it doesn’t use.
- **Personal data:** the only personal data is task assignee names. There are no emails, HR data or customer personal data. Outputs never judge individuals’ performance.
- **Vendors:** OpenAI API (no training on API data under its standard terms; add a data-processing agreement and zero-retention if the client requires it) and Langfuse (EU region, 30-day retention on the Hobby tier; self-host if the client contract requires it). Both go on the DPO’s register for UK GDPR.
- **Client confidentiality:** ABCDE’s data is processed only for ABCDE’s project, and no cross-client data appears in prompts. When there are multiple projects, access follows Nexora’s project membership.
- **Secrets:** API keys live only in n8n credentials, never in workflow JSON or Git (the exported workflow contains no credentials).

5. Rollout to 10 PMs/TPMs

Week	Step
1	Pick 2 champion PMs; shadow mode on the ABCDE project, with Mira and manual reports side by side
2	Fix the top issues from traces and feedback; re-run the regression suite
3	Onboard 8 more PMs (30-minute session plus a one-page prompt guide); Slack front end live; daily alerts on
4–6	Weekly review of the Langfuse dashboard (groundedness, refusals, cost, latency) and time-saved survey
7	Go/no-go against the charter's gates: ≥ 98% grounded, 100% count accuracy, ≥ 8/10 weekly active PMs, ≥ 60 min saved per PM per week

Personal takeaway: the most valuable design decision wasn't a prompt. It was deciding what the LLM should *not* do. Code computes, the model explains, and the system asks when it doesn't know.